Próximos pasos

Corridas para la máquina de 32 GB

Capstone ICS2122 — Grupo 9

10 de noviembre de 2026 (actualizado con interchanges)

En una página

Optimizamos el itinerario semanal de una flota de carga de 19 aviones: qué vuelos hace cada avión, a qué hora, cuánta carga lleva y dónde la transborda. El objetivo es maximizar el margen (ingresos menos costos).

- **Cambio importante (9 y 10 de noviembre):** el profesor confirmó que se permiten los *interchanges*: un avión puede pasar a ser operado por otra filial a mitad de semana. Las filiales cobran muy distinto por hora de vuelo (PAC 3.300 USD/h, AND 4.450 USD/h), así que esto cambia mucho el margen.
- **Mejor resultado actual: 7.406.840 USD**, validado, con interchanges y borde flexible. Está en `data/results/Interchange_continua_flexible`.
- **Detén las corridas que tengas sin `--interchange`** (Ctrl+C; la mejor solución ya quedó guardada). Eligen vuelos con los costos antiguos, así que su resultado queda muy por debajo.
- El Paso 1 que ya corriste (`RamaB_red_dt1`, 6.407.941) **sí sirve**: el Paso 3 le agrega los interchanges.
- **Tu tarea:** 3 corridas, una por noche. Todos los comandos están en `docs/comandos_32gb.txt`. Cópialos desde ahí y no desde el PDF, porque el PDF corta las líneas largas.

Tabla 1: Cómo llegamos al mejor resultado (margen validado, USD).

Etapa	Margen	Aporte
Heurística con grilla de 2 h	6.310.378	—
Heurística con grilla de 1 h y punto de partida	6.336.259	+25.881
MILP integrado en tiempo continuo, con NoRel	6.435.839	+99.580
Búsqueda por vecindarios: un avión a la vez	6.451.322	+15.483
Búsqueda por vecindarios: ventanas de 12 h	6.466.083	+14.761
Ventanas con desplazamiento (agregar o quitar vuelos)	6.475.055	+8.972
Interchanges sobre los mismos vuelos (borde base)	7.273.495	+798.440
Búsqueda con interchanges, 2,4 h (borde base)	7.308.301	+34.806
Borde flexible sobre esos vuelos	**7.406.840**	+98.539

1. Conceptos que necesitas

Margen validado
El único número que comparamos. Toma los vuelos de una solución, recalcula la carga con un modelo exacto (gap 10^{-6}) y revisa todas las reglas con el validador. Las búsquedas solo aceptan soluciones que pasan el validador.

Interchange
Un avión pasa a ser operado por otra filial en un aeropuerto donde ambas están habilitadas (`interchange_airports.csv`). Cada vuelo usa los derechos de tráfico, el costo por hora y el handling de la filial que lo opera. Detalle en `docs/README_interchanges.md`.

Borde flexible
Como la semana se repite, el avión debe volver a su filial base. Usamos la lectura flexible: basta con que vuelva en tierra, en el aeropuerto donde parte y termina la semana, si allí están habilitadas ambas filiales. Es el valor por defecto; `--borde base` exige en cambio que el primer y el último vuelo sean de la filial base.

Red espacio-tiempo
La semana se divide en bloques de DT horas (aquí, 1 h). Cada vuelo es un arco entre dos pares (aeropuerto, bloque). Su relajación es ajustada, pero redondea hacia arriba la duración de cada vuelo, así que pierde minutos.

MILP en tiempo continuo
Cada avión tiene posiciones $1, \ldots, 32$ y cada posición lleva un vuelo con su horario exacto. No pierde minutos, pero su relajación es débil (gap de $\sim$30 %).

Vecindario
Una parte de las decisiones que se deja libre mientras el resto queda fijo. Puede ser un avión, un par o un trío de aviones, un operador completo o una ventana de tiempo. Se resuelve el MILP exacto con la carga y los transbordos de *toda* la red, y el cambio se acepta solo si sube el margen validado.

Ventana de tiempo
Libera todos los vuelos que salen entre t_0 y t_1, en todos los aviones a la vez. Permite reorganizar varios aviones juntos.

Desplazamiento
Permite que una ventana gane o pierda hasta D vuelos: los vuelos posteriores mantienen tramo y horario, pero se corren de posición. Aprovecha los aviones que tienen 20–76 h libres en la semana.

NoRel
Heurística de Gurobi que busca soluciones sin resolver la relajación. Sirve cuando la raíz es muy cara, como en nuestro modelo completo.

2. Por qué hace falta tu computador

Cada método crece con el tamaño de lo que se deja libre. La tabla muestra lo que se estimó con `--estimar`:

En el notebook, las ventanas de 24 h no terminaron la raíz en el tiempo dado, y una corrida de 12 h bloquea el computador. Con 32 GB y más núcleos, las corridas largas caben y no frenan el resto del trabajo.

Tabla 2: Memoria aproximada de la red espacio-tiempo por vecindario (instancia completa).

Vecindario	DT	Variables	RAM
1 avión	1 h	1,9 M	8 GB (justo)
2 aviones	1 h	3,2 M	16 GB
4 aviones (operadores BRA y PAC)	1 h	5,2 M	32 GB
8 aviones (operador AND)	1 h	10,6 M	64 GB
Todos los aviones	1 h	—	no cabe

3. El plan

Tabla 3: Las corridas, en orden. Corre una a la vez: cada una usa todos los núcleos.

Paso	Duración	Qué hace	Parte de
0	5 min	Detener lo que corre y actualizar el código	—
1	*hecho*	Red espacio-tiempo, aviones de a uno y de a dos (6.407.941)	heurística
2	noche (12 h)	Tiempo continuo con interchanges, ventanas de 12 h	mejor actual
3	10 min + noche	Rama B: interchanges sobre el Paso 1 y luego tiempo continuo	resultado del Paso 1
4	noche (12 h)	Vecindarios grandes con interchanges: ventanas de 24 h, tríos, operadores	mejor resultado
5	10 min	Revisar y subir los resultados	—

Los pasos 2 y 3 son independientes. El Paso 4 parte del mejor resultado que haya en ese momento.

El punto de partida decide a qué óptimo local se llega. En la instancia reducida R4, la rama B (heurística → red espacio-tiempo → tiempo continuo) llegó a 868.027, y la rama A (la del mejor actual) a 839.809. Por eso el Paso 3 corre la rama B con interchanges.

4. Paso 0 — Detener y actualizar

1. Si tienes una corrida en curso sin `--interchange`, detenla con Ctrl+C. Su mejor solución ya está en la carpeta de `--salida`.
2. Actualiza la rama (tus carpetas de resultados no se tocan):

   ```
   git pull
   ```

3. Deja el computador enchufado. En Windows, el script evita la suspensión por sí solo. En Mac, antepone `caffeinate -i` a cada comando; en Linux, `systemd-inhibit`. En Mac o Linux, cambia `python` por `python3`.

Cómo saber si salió bien: existe la carpeta `data/results/Interchange_continua_flexible`.

5. Paso 2 — Tiempo continuo con interchanges desde el mejor actual

Qué hace: la misma búsqueda que dio los mejores resultados (ventanas de 12 h con desplazamiento, aviones y pares), ahora con `--interchange`. Los modelos eligen los vuelos con el costo de la filial más barata con derechos. Cada candidato se evalúa asignando las filiales reales y la carga, y se acepta solo si sube el margen validado.

Por qué: en el notebook esta búsqueda subió +34.806 en 2,4 h (con borde base) y seguía encontrando mejoras cuando se detuvo.

```
python -u busqueda_por_avion.py --modelo continuo --interchange --corrida
    data/results/Interchange_continua_flexible --vecindario ventana,avion,par
    --ventana-h 12 --ventana-paso 6 --desplazar 2 --desplazar-aviones 10
    --desplazar-presupuesto 60 --tiempo 300 --norel 60 --saltar-sin-cambios
    --umbral-aceptar 100 --mejora-minima 2000 --orden uso --memoria 24 --vueltas 20
    --horas 12 --salida data/results/Interchange_ventanas 2>&1 | tee
    data/results/Interchange_ventanas.log
```

Qué esperar: la primera línea incluye `interchanges, borde flexible` y `margen validado 7,406,840 USD`. Cada ventana toma de 30 s a 8 min, porque evaluar un candidato resuelve la carga hasta 3 veces. Una pasada por las 28 ventanas toma ∼1,5–2 h.

Cómo saber si salió bien: el margen final supera 7.406.840.

6. Paso 3 — Rama B con interchanges

Qué hace: primero asigna las filiales a los vuelos de tu Paso 1 (programación dinámica exacta, unos minutos). Después pule esos vuelos en tiempo continuo con interchanges.

Por qué: el Paso 1 se hizo con los costos antiguos, pero sus vuelos son otro punto de partida. En R4, la rama B ganó.

```
python -u reasignar_operadores.py --corrida data/results/RamaB_red_dt1 --salida
    data/results/RamaB_interchange 2>&1 | tee data/results/RamaB_interchange.log
python -u busqueda_por_avion.py --modelo continuo --interchange --corrida
    data/results/RamaB_interchange --vecindario ventana,avion,par --ventana-h 12
    --ventana-paso 6 --desplazar 2 --desplazar-aviones 10 --desplazar-presupuesto 60
    --tiempo 300 --norel 60 --saltar-sin-cambios --umbral-aceptar 100 --mejora-minima
    2000 --orden uso --memoria 24 --vueltas 20 --horas 12 --salida
    data/results/RamaB_interchange_continua 2>&1 | tee
    data/results/RamaB_interchange_continua.log
```

Qué esperar: el primer comando termina con `Margen validado: 6,407,941 -> X USD`. El segundo se comporta como el Paso 2.

Cómo saber si salió bien: si el margen final supera el del Paso 2, la rama B también gana con interchanges.

7. Paso 4 — Vecindarios grandes con interchanges

Qué hace: parte del mejor resultado de los pasos 2 y 3 y prueba vecindarios más grandes que los que caben en el notebook: ventanas de 24 h, tríos de aviones y operadores completos.

Por qué: cuando las ventanas de 12 h dejan de mejorar, la solución es un óptimo local para ellas. Para salir de ahí hacen falta vecindarios más grandes, y esos necesitan memoria.

```
python -u busqueda_por_avion.py --modelo continuo --interchange --corrida
    data/results/Interchange_ventanas --vecindario ventana,trio,operador --ventana-h
    24 --ventana-paso 12 --desplazar 2 --desplazar-aviones 10 --desplazar-presupuesto
    60 --tiempo 900 --norel 120 --saltar-sin-cambios --umbral-aceptar 100
    --mejora-minima 2000 --orden uso --memoria 24 --vueltas 20 --horas 12 --salida
    data/results/Interchange_grandes 2>&1 | tee data/results/Interchange_grandes.log
```

Antes de correrlo: si el Paso 3 terminó con más margen que el Paso 2, cambia `Interchange_ventanas` por `RamaB_interchange_continua` en `--corrida`.

Qué esperar: hasta 15 min por vecindario. Es posible que no mejore; el resultado igual sirve, porque muestra que el óptimo local resiste vecindarios grandes.

8. Paso 5 — Revisar y subir

1. Revisa la última parte de cada `.log`: termina con `Margen validado: A -> B USD`, donde A es el margen de partida y B el final. No imprime «Validador: OK», porque cada solución aceptada ya pasó el validador.
2. Para confirmarlo, abre `indicadores_kpi.json` en la carpeta de salida: debe decir `"validador_ok": true`.
3. En la carpeta de salida, `progreso.csv` tiene una fila por vecindario. La columna `aceptado` marca las mejoras, y `mejor_margen` muestra cómo fue subiendo el margen.
4. Sube los resultados a la misma rama, nunca a `main`:

   ```
   git add data/results/RamaB_* data/results/Interchange_ventanas*
       data/results/Interchange_grandes*
   git commit -m "resultados maquina 32 GB con interchanges"
   git pull
   git push
   ```

5. Avisa al grupo el margen final de cada paso.

9. Las opciones de los comandos

10. Si algo falla

11. Qué no vale la pena correr

- **Ramificar el modelo completo hasta el óptimo:** su relajación es débil (gap de 42–47 % en R4), y más memoria no lo arregla.
- **Benders:** es correcto, pero no converge en la práctica.
- **Una segunda pasada del MILP integrado después de la búsqueda continua:** empeora el margen con transbordos.
- **Grillas de 30 min:** aportaron ∼0,1 %.
- **Cualquier corrida sin `--interchange`:** elige vuelos con los costos antiguos.

Opción	Qué controla
`--corrida`	Carpeta de la solución de partida.
`--salida`	Carpeta donde se guarda la mejor solución; se actualiza con cada mejora.
`--modelo continuo`	Usa el MILP en tiempo continuo; sin esta opción, la red espacio-tiempo.
`--dt`	Horas por bloque de la red espacio-tiempo.
`--vecindario`	Vecindarios a alternar: `avion`, `par`, `trio`, `operador`, `ventana`.
`--tiempo`, `--norel`	Segundos de Gurobi por vecindario, y cuántos de ellos se dedican a NoRel.
`--memoria`	GB máximos para Gurobi; con 32 GB, 24 deja espacio al sistema.
`--hilos`	Núcleos de Gurobi (0 = todos).
`--horas`, `--vueltas`	Detienen la búsqueda tras esas horas o vueltas completas.
`--mejora-minima`	Una mejora menor que esto (USD) no hace volver al primer vecindario.
`--umbral-aceptar`	Mejora mínima (USD) para aceptar un cambio; evita cambios casi equivalentes.
`--saltar-sin-cambios`	Salta un vecindario que ya falló, mientras su región siga igual.
`--ventana-h`, `--ventana-paso`	Ancho de la ventana y cuánto avanza cada vez (h).
`--desplazar`	Vuelos que cada avión puede ganar o perder dentro de la ventana.
`--desplazar-aviones`	Cuántos aviones pueden desplazarse (los con más horas libres).
`--desplazar-presupuesto`	Tope total de vuelos que se corren; sin él, el modelo explota.
`--orden uso`, `--max-pares`	Recorre primero los aviones más usados; limita cuántos pares se prueban.
`--interchange`	Permite que otra filial opere cada vuelo; sin esta opción, cada avión vuela siempre con su filial base.
`--borde`	Borde de la semana con interchanges: `flexible` (por defecto) o `base`.

- **El MILP integrado (`warm_start_milp.py`), por ahora:** todavía no está adaptado ni probado con interchanges.

12. Respaldo en la literatura

Cada método que usamos tiene una base estándar en la literatura:

- **Modelo integrado** (rama A, de donde sale el mejor resultado). En aerolíneas de carga, la selección de vuelos, las rotaciones y el ruteo de la carga dependen entre sí, y planificarlos por etapas pierde margen. Derigs et al. proponen modelos que los deciden juntos [4, 3].
- **Red espacio-tiempo** (Paso 1). Es el modelo clásico de asignación de flota, como flujo multiproducto en una red expandida en el tiempo [6].
- **Tiempo continuo después de la grilla** (pasos 2–4). Discretizar el tiempo puede excluir las mejores soluciones; Boland et al. muestran que el óptimo en tiempo continuo se alcanza refinando la discretización solo donde hace falta [1]. Eso explica por qué pulir en tiempo continuo la solución de la grilla sube el margen.
- **Búsqueda por vecindarios exactos** (todos los pasos). Liberar una parte de la solución, fijar el resto y reoptimizarla es una búsqueda en vecindarios grandes, LNS [10, 9]. Cuando cada vecindario se resuelve con un MILP sobre un subconjunto de binarias, es un esquema de tipo *fix-and-optimize* [7], emparentado con los vecindarios definidos con un MILP, como RINS [2].
- **Alternar vecindarios** (avión, par, trío, ventana). Es la idea de la búsqueda de vecindad

Síntoma	Qué hacer
Model too large for size-limited license	Falta la licencia académica: repite el Paso 0.1.
Out of memory o el PC se congela	Cambia `--memoria 24` por `--memoria 16`. Si sigue, en el Paso 4 quita `,trio`.
Se cortó (luz, reinicio, Ctrl+C)	La mejor solución ya está guardada en `--salida`. Repite el comando con `--corrida` igual a esa carpeta.
La primera línea no dice *interchanges, borde flexible*	Falta `--interchange` en el comando, o falta el `git pull`.
Una noche sin mejoras	Es un resultado válido: súbelo igual y pasa al paso siguiente.

variable, VNS [8].

- **NoRel con punto de partida.** Gurobi documenta esta heurística como útil cuando la relajación de la raíz es muy cara [5].

Nuestra evidencia propia: en R4, la búsqueda por vecindarios ganó al modelo completo (+8,6 % contra +5,3 %). Eso respalda usar LNS en un modelo cuya relajación es débil.

Referencias

[1] N. Boland, M. Hewitt, L. Marshall y M. Savelsbergh (2017). The continuous-time service network design problem. *Operations Research*, 65(5), 1303–1321. doi:10.1287/opre.2017.1624

[2] E. Danna, E. Rothberg y C. Le Pape (2005). Exploring relaxation induced neighborhoods to improve MIP solutions. *Mathematical Programming*, 102(1), 71–90. doi:10.1007/s10107-004-0518-7

[3] U. Derigs y S. Friederichs (2013). Air cargo scheduling: integrated models and solution procedures. *OR Spectrum*, 35(2), 325–362. doi:10.1007/s00291-012-0299-y

[4] U. Derigs, S. Friederichs y S. Schäfer (2009). A new approach for air cargo network planning. *Transportation Science*, 43(3), 370–380. doi:10.1287/trsc.1090.0282

[5] Gurobi Optimization (2026). *Gurobi Optimizer Reference Manual*, parámetro NoRelHeurTime. `https://docs.gurobi.com/projects/optimizer/en/current/reference/parameters.html`

[6] C. A. Hane, C. Barnhart, E. L. Johnson, R. E. Marsten, G. L. Nemhauser y G. Sigismondi (1995). The fleet assignment problem: solving a large-scale integer program. *Mathematical Programming*, 70, 211–232. doi:10.1007/BF01585938

[7] S. Helber y F. Sahling (2010). A fix-and-optimize approach for the multi-level capacitated lot sizing problem. *International Journal of Production Economics*, 123(2), 247–256. `https://econpapers.repec.org/RePEc:eee:proeco:v:123:y:2010:i:2:p:247-256`

[8] N. Mladenović y P. Hansen (1997). Variable neighborhood search. *Computers & Operations Research*, 24(11), 1097–1100. doi:10.1016/S0305-0548(97)00031-2

[9] D. Pisinger y S. Ropke (2010). Large neighborhood search. En M. Gendreau y J.-Y. Potvin (eds.), *Handbook of Metaheuristics*, 2.ª ed., 399–419. Springer. doi:10.1007/978-1-4419-1665-5_13

[10] P. Shaw (1998). Using constraint programming and local search methods to solve vehicle routing problems. En *Principles and Practice of Constraint Programming (CP98)*, LNCS 1520, 417–431. Springer. doi:10.1007/3-540-49481-2_30